PRAKTIKUM ALGORITMA DAN STRUKTUR DATA
JOBSHEET 11 LINKED LIST

NAMA : LUCKY SATRIA UTAMA
NIM : 254107060142
JURUSAN : TEKNOLOGI INFORMASI
PRODI : D-IV SISTEM INFORMASI BISNIS
KELAS : SIB – 1F

TAHUN 2025/2026

Percobaan 1:
Mahasiswa09.java

```java
package Pertemuan12;

public class Mahasiswa09 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa09() {}

    public Mahasiswa09(String nm, String name, String kls, double ip) {
        this.nim = nm;
        this.nama = name;
        this.kelas = kls;
        this.ipk = ip;
    }

    public void tampilInformasi() {
        System.out.printf("%-15s %-12s %-5s %.1f%n", nama, nim, kelas, ipk);
    }
}
```

NodeMahasiswa09.java

```java
package Pertemuan12;

public class NodeMahasiswa09 {
    Mahasiswa09 data;
    NodeMahasiswa09 next;

    public NodeMahasiswa09(Mahasiswa09 data, NodeMahasiswa09 next) {
        this.data = data;
        this.next = next;
    }
}
```

SingleLinkedList09.java

```java
package Pertemuan12;

public class SingleLinkedList09 {
    NodeMahasiswa09 head;
    NodeMahasiswa09 tail;

    boolean isEmpty() {
        return (head == null);
```

```java
    }

    public void print() {
        if (!isEmpty()) {
            NodeMahasiswa09 tmp = head;
            System.out.print("Isi Linked List:\t");
            System.out.println();
            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        } else {
            System.out.println("Linked list kosong");
        }
    }

    public void addFirst(Mahasiswa09 input) {
        NodeMahasiswa09 ndInput = new NodeMahasiswa09(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }

    public void addLast(Mahasiswa09 input) {
        NodeMahasiswa09 ndInput = new NodeMahasiswa09(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            tail.next = ndInput;
            tail = ndInput;
        }
    }

    public void insertAfter(String key, Mahasiswa09 input) {
        NodeMahasiswa09 ndInput = new NodeMahasiswa09(input, null);
        NodeMahasiswa09 temp = head;
        do {
            if (temp.data.nama.equalsIgnoreCase(key)) {
                ndInput.next = temp.next;
                temp.next = ndInput;
                if (ndInput.next == null) {
                    tail = ndInput;
```

```java
                }
                break;
            }
            temp = temp.next;
        } while (temp != null);
    }

    public void insertAt(int index, Mahasiswa09 input) {
        if (index < 0) {
            System.out.println("indeks salah");
        } else if (index == 0) {
            addFirst(input);
        } else {
            NodeMahasiswa09 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = new NodeMahasiswa09(input, temp.next);
            if (temp.next.next == null) {
                tail = temp.next;
            }
        }
    }
}
```

SLLMain09.java

```java
package Pertemuan12;

public class SLLMain09 {
    public static void main(String[] args) {
        SingleLinkedList09 sll = new SingleLinkedList09();

        Mahasiswa09 mhs1 = new Mahasiswa09("24212200", "Alvaro", "1A", 4.0);
        Mahasiswa09 mhs2 = new Mahasiswa09("22212202", "Cintia", "3C", 3.5);
        Mahasiswa09 mhs3 = new Mahasiswa09("23212201", "Bimon",  "2B", 3.8);
        Mahasiswa09 mhs4 = new Mahasiswa09("21212203", "Dirga",  "4D", 3.6);

        sll.print();                        // kosong
        sll.addFirst(mhs4);
        sll.print();                        // Dirga
        sll.addLast(mhs1);
        sll.print();                        // Dirga, Alvaro
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();                        // Dirga, Cintia, Bimon, Alvaro
    }
}
```

Pertanyaan:

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”? Karena perintah pertama yang dijalankan adalah sll.print() **sebelum** ada data yang ditambahkan ke linked list. Pada saat itu, nilai head masih null karena belum ada node yang dibuat. Method isEmpty() mengecek head == null dan hasilnya true, sehingga program mencetak **"Linked list kosong"**.

2. Jelaskan kegunaan variable temp secara umum pada setiap method! Variable temp berfungsi sebagai **pointer sementara** untuk melakukan traversal (penelusuran) pada linked list **tanpa mengubah posisi head asli**. Jika langsung menggunakan head untuk traversal, maka head akan bergeser dan linked list akan kehilangan referensi ke node pertama. Dengan temp, penelusuran bisa dilakukan dari awal hingga akhir list dengan aman.

3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!

```java
package Pertemuan12;
import java.util.Scanner;

public class SLLMain09 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList09 sll = new SingleLinkedList09();

        System.out.print("Berapa mahasiswa yang ingin ditambahkan? ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.println("\n--- Data Mahasiswa ke-" + (i+1) + " ---");
            System.out.print("NIM   : ");
            String nim = sc.nextLine();
            System.out.print("Nama  : ");
            String nama = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa09 mhs = new Mahasiswa09(nim, nama, kelas, ipk);
            sll.addLast(mhs);
            sll.print();
        }

        sc.close();
    }
}
```

Percobaan 2:

SingleLinkedList09.java

```
// ===== PERCOBAAN 2 =====

    public void getData(int index) {
        NodeMahasiswa09 tmp = head;
        for (int i = 0; i < index; i++) {
            tmp = tmp.next;
        }
        tmp.data.tampilInformasi();
    }

    public int indexOf(String key) {
        NodeMahasiswa09 tmp = head;
        int index = 0;
        while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
            tmp = tmp.next;
            index++;
        }
        if (tmp == null) {
            return -1;
        } else {
            return index;
        }
    }

    public void removeFirst() {
        if (isEmpty()) {
            System.out.println("Linked List masih Kosong, tidak dapat
dihapus!");
        } else if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
        }
    }

    public void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List masih Kosong, tidak dapat
dihapus!");
        } else if (head == tail) {
            head = tail = null;
        } else {
            NodeMahasiswa09 temp = head;
            while (temp.next != tail) {
                temp = temp.next;
```

```
            }
            temp.next = null;
            tail = temp;
        }
    }

    public void remove(String key) {
        if (isEmpty()) {
            System.out.println("Linked List masih Kosong, tidak dapat
dihapus!");
        } else {
            NodeMahasiswa09 temp = head;
            while (temp != null) {
                if ((temp.data.nama.equalsIgnoreCase(key)) && (temp == head))
{
                    this.removeFirst();
                    break;
                } else if (temp.data.nama.equalsIgnoreCase(key)) {
                    temp.next = temp.next.next;
                    if (temp.next == null) {
                        tail = temp;
                    }
                    break;
                }
                temp = temp.next;
            }
        }
    }

    public void removeAt(int index) {
        if (index == 0) {
            removeFirst();
        } else {
            NodeMahasiswa09 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = temp.next.next;
            if (temp.next == null) {
                tail = temp;
            }
        }
    }
}
```

SLLMain09.java

```
package Pertemuan12;
```

```java
import java.util.Scanner;

public class SLLMain09 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList09 sll = new SingleLinkedList09();

        System.out.print("Berapa mahasiswa yang ingin ditambahkan? ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.println("\n--- Data Mahasiswa ke-" + (i+1) + " ---");
            System.out.print("NIM   : ");
            String nim = sc.nextLine();
            System.out.print("Nama  : ");
            String nama = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
            System.out.print("IPK   : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa09 mhs = new Mahasiswa09(nim, nama, kelas, ipk);
            sll.addLast(mhs);
            sll.print();
        }

        // Percobaan 2
        System.out.println("data index 1 : ");
        sll.getData(1);

        System.out.println("data mahasiswa an Bimon berada pada index : "
        + sll.indexOf("bimon"));
        System.out.println();

        sll.removeFirst();
        sll.removeLast();
        sll.print();
        sll.removeAt(0);
        sll.print();

        sc.close();
    }
}
```

Pertanyaan:

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan! Keyword break digunakan untuk **menghentikan perulangan while** setelah node yang dicari ditemukan dan dihapus. Tanpa break, perulangan akan terus berjalan meskipun node sudah dihapus, yang bisa menyebabkan error karena program tetap mencoba mengakses node berikutnya yang sudah tidak valid. Jadi break memastikan proses penghapusan hanya terjadi **satu kali** pada node yang sesuai lalu langsung keluar dari loop.
2. Jelaskan kegunaan kode dibawah pada method remove

```
1  temp.next = temp.next.next;
2  if (temp.next == null) {
3      tail = temp;
4  }
```

Baris 1: temp.next = temp.next.next Berfungsi untuk **menghapus node** dengan cara memotong referensi. Node sebelumnya (temp) langsung dihubungkan ke node dua langkah ke depan, sehingga node yang dihapus tidak lagi terhubung ke linked list.

Baris 2-4: if (temp.next == null) tail = temp Berfungsi untuk **memperbarui posisi tail** jika node yang dihapus ternyata adalah node terakhir. Jika setelah penghapusan temp.next bernilai null, berarti temp sekarang menjadi node terakhir, sehingga tail diupdate ke temp.